The Complexity of Boolean Connectivity Problem of k-Horn Formulas

Takashi Horiyama[1], Shoon Mineyoshi[1], Yuto Okura[1], Kazuhisa Seto[1], and Junichi Teruyama[2]

[1]Hokkaido University, {horiyama,seto}@ist.hokudai.ac.jp
[2]University of Hyogo, junichi.teruyama@gsis.u-hyogo.ac.jp

Abstract

The Boolean connectivity problem asks whether the set of satisfying assignments of a given Boolean formula forms a connected subgraph in the n-dimensional hypercube. This problem is known to be coNP-complete, even when restricted to k-Horn formulas for $k \geq 3$, as shown by Makino, Tamaki, and Yamamoto. In this paper, we further investigate the computational complexity of Conn k-Horn, the Boolean connectivity problem for k-Horn formulas. We provide algorithmic and hardness results for Conn k-Horn. On the algorithmic side, we first present an exact exponential-time algorithm for arbitrary k without any structural restrictions. Our algorithm builds on the deterministic PPZ algorithm proposed by Paturi, Pudlák, and Zane. It runs in $O^*(2^{(1-1/2k)n})$ time and polynomial space, achieving an exponential improvement over the previously known algorithm for the Boolean connectivity problem of k-CNF formulas, shown by Makino, Tamaki, and Yamamoto. We next give two polynomial-time algorithms for arbitrary k under the following two restrictions: (i) each variable appears at most twice, and (ii) each clause has length exactly k and each variable appears at most k times. On the hardness side, we prove that Conn 3-Horn remains coNP-complete even when each variable appears exactly three times.

1 Introduction

The Boolean connectivity problem asks whether the satisfying assignments of a given Boolean formula are connected over the n-dimensional hypercube. The structure and connectivity of satisfying assignments are closely related to the analysis of satisfiability algorithms and satisfiability thresholds. The satisfiability problem is characterized by the famous Schaefer dichotomy theorem [10]. Schaefer's class denotes the class of formulas, e.g., 2-CNF, (dual) Horn, and affine formulas. Ekin, Hammer, and Kogan [4] showed that the Boolean connectivity of DNF is solvable in time linear in the size of DNF, while determining the connectivity of falsifying assignments of DNF is coNP-hard, i.e., the Boolean connectivity of CNF is coNP-hard. Gopalan, Kolaitis, Maneva, and Papadimitriou [5] extensively studied the computational complexity of the Boolean connectivity problem. The paper [5] showed that it is either coNP-complete or PSPACE-complete for non-Schaefer's class, and in coNP for Schaefer's class. It has also been conjectured that the problem admits a polynomial-time algorithm for Schaefer's class. Makino, Tamaki, and Yamamoto [6] and Schwerdtfeger [13] gave a negative answer by proving the coNP-completeness of the Boolean connectivity problem for k-Horn and other related formulas.

Although the trichotomy theorem of the Boolean connectivity problem has been fully established, only a few algorithms are known for instances beyond P. For the connectivity of k-CNF formulas (Conn k-CNF), it is solvable in polynomial time when $k = 2$ [5]. Makino et al. [7] presented a moderately exponential-time and exponential-apce algorithm for Conn k-CNF ($k \geq 3$) over n variables and m clauses which is PSPACE-complete. Their algorithm constructs a solution graph over partial satisfying assignments and subsequently determines its connectivity. It runs in $O^*(2^{(1-c_k)n})$ time and exponential space, where $c_k = \log \beta_k/(1 + \log \beta_k)$ and β_k (< 2) is the largest positive real number that satisfies $x^k - x^{k-1} - \cdots - x - 1 = 0$. The O^* notation suppresses polynomial factors in n and m. Throughout this paper, the base of the logarithm is 2. Note that $c_k = 1/2^{O(k)}$.

In this paper, we deeply investigate the complexity of the Boolean connectivity of k-Horn formulas (Conn k-Horn). We first consider an exact algorithm for Conn k-Horn without any structural

restrictions. The algorithm of Makino et al. can also solve Conn k-Horn in the same running time since k-Horn formulas are subsets of k-CNF formulas. Consequently, it is natural to consider whether a faster algorithm for Conn k-Horn exists. Makino et al. [6] proposed an exact algorithm that runs in polynomial time with respect to the number of variables and the size of the characteristic set of a Horn formula. Unfortunately, the efficiency of computing the characteristic set of a given Horn formula is not yet well understood. We give an affirmative answer by utilizing the deterministic PPZ algorithm for k-SAT shown by Paturi, Pudlák, and Zane [9]. The deterministic PPZ algorithm can find not only a satisfiability assignment but also all locally minimal satisfying assignments. Moreover, Conn k-Horn can be solved to find a locally minimal satisfying assignment with Hamming weight at least one [5]. These observations enable us to solve Conn k-Horn exponentially faster than algorithms for Conn k-CNF. Moreover, the proposed algorithm requires only polynomial space.

Theorem 1. *Given a k-Horn formula over n variables, there exists a deterministic $O^*(2^{(1-1/2k)n})$ time and polynomial-space algorithm for* Conn k-Horn.

We next consider Conn k-Horn with a bounded number of occurrences of each variable. We present the following algorithmic and hardness results.

- A polynomial-time algorithm for Conn k-Horn when each variable appears at most twice (Conn k-Horn-2). The algorithm is based on the relationship between variables that appear only as negative literals and locally minimal satisfying assignments. Any variable that appears only as negative literals is assigned the value 0 in any locally minimal satisfying assignment. We prove that determining the connectivity of the solution graph for the remaining formula after assigning 0 to such variables is solvable in polynomial time.
- A polynomial-time algorithm for Conn k-Horn when each clause have exactly k literals and each variable appears at most k times (Conn Ek-Horn-k). The algorithm is based on the relationship between the solution graph of a Horn formula and a non-empty maximal self-implicating set, as shown in [13]. Any variable in a self-implicating set is forced to be true when all the other variables in the set are true. The solution graph is disconnected if and only if there exists a non-empty maximal self-implicating set and no clauses with only negative literals of these variables. We show that such a set can be found in polynomial time.
- The coNP-completeness of Conn 3-Horn even when each variable appears exactly three times (Conn 3-Horn-E3). We first prove the complement of Conn 3-Horn remains NP-complete even if each variable appears at most three times (Conn 3-Horn-3) by a polynomial-time reduction from Monotone Not-All-Equal 3-SAT, where each clause has exactly three literals and each variable appears exactly four times (Monotone NAE E3-SAT-E4). This problem is known to be NP-complete [3]. We then reduce the Conn 3-Horn-3 to the Conn 3-Horn-E3.

Related Work The connectivity problem between two satisfying assignments (st-Conn) has also been extensively studied. Gopalan et al. [5] showed that st-Conn is solvable in polynomial time for Schaefer's and a part of non-Schaefer's class and is PSPACE-complete otherwise. Scharpfenecker [11] proved that the complexity of SAT is equivalent to that of st-Conn in Schaefer's class. This implies that st-Conn is P-complete for Horn formulas and is NL-complete for 2-CNFs. Cardinal, Demaine, Eppstein, Hearn, and Winslow [2] showed that st-Conn of Planar Monotone Not-All-Equal 3-SAT is PSPACE-complete.

The problem of finding the shortest path in the solution graph of Boolean formulas has also been investigated. Mouawad, Nishimura, Pathak, and Raman [8] studied the computational complexity of this problem and proved a trichotomy theorem showing that the problem lies in P, is NP-complete, or is PSPACE-complete. The paper [8] also showed that there exist classes of Boolean formulas for which the shortest path can be found in polynomial time, even though its length is not equal to the symmetric difference between the values of the variables in s and t. Bonsma, Mouawad, Nishimura, and Raman [1] showed the problem is W[1]-hard when parameterized by the path length ℓ from s to t.

The complexity of the isomorphism of two solution graphs was studied in [12]. This problem is PSPACE-hard and lies in EXP for general formulas, and is $\mathsf{C}_=$P-complete for 2-CNF. The class $\mathsf{C}_=$P is a subclass of PSPACE and is defined in terms of exact counting.

2 Preliminaries

Let x be a Boolean variable that takes true (1) or false (0). A *literal* is a Boolean variable x (positive literal) or its negation $\bar{x}$ (negative literal). A *clause* is a disjunction of literals. The *length* of clause

C is defined as the number of literals in C. A *unit clause* is a clause of length one. A conjunctive normal form (CNF) formula is a conjunction of clauses, and a k-CNF formula is a CNF formula if the length of each clause is at most k. A CNF formula is *monotone* if all clauses in the formula consist of only positive (or negative) literals. In this paper, we use Monotone CNF as CNF formulas with all positive literals unless otherwise stated. A *Horn clause* is a clause including at most one positive literal. A Horn formula is a CNF formula whose all clauses are Horn, and a k-Horn formula is a k-CNF formula and a Horn formula. Let $X = \{x_1, x_2, ..., x_n\}$ be a set of Boolean variables. An *assignment* of a Boolean formula φ over X is $\alpha = (\alpha_1, \alpha_2, \ldots, \alpha_n) \in \{0,1\}^n$ such that $x_i = \alpha_i$ for all i. A *partial assignment* of a Boolean formula φ over X is $\alpha = (\alpha_1, \alpha_2, \ldots, \alpha_n) \in \{0,1,*\}^n$ such that $x_i = \alpha_i$ for all i. A partial assignment with $\alpha_i = *$ means variable x_i is not assigned a $0/1$ value. For a variable x and an assignment α, we denote by $\alpha(x)$ the value of x under an assignment α. We can simplify a Boolean formula φ by a partial assignment α in a natural way, and the resulting formula is denoted by $\varphi|_\alpha$. An (partial) assignment α is a *(partial) satisfying assignment* of a Boolean formula φ if $\varphi|_\alpha$ is true. The satisfiability problem is to determine whether there exists a satisfying assignment of a given Boolean formula φ.

Many excellent exponential-time algorithms for the satisfiability problem of k-CNF formulas (k-CNF SAT) are known. One of the famous algorithms is shown by Paturi, Pudlák, and Zane [9].

Theorem 2 ([9]). *There exists an algorithm that can solve k-CNF SAT over n variables in a deterministic $O^*(2^{(1-1/2k)n})$ time and polynomial space.*

For an assignment $\alpha \in \{0,1\}^n$, the *Hamming weight* of α is $|\{i : \alpha_i = 1 \ (1 \le i \le n)\}|$. For two assignments $\alpha, \alpha' \in \{0,1\}^n$, the *Hamming distance* between α and α', denoted by $d(\alpha, \alpha')$, is $|\{i : \alpha_i \neq \alpha'_i \ (1 \le i \le n)\}|$. We define the *solution graph* of a Boolean formula φ as $G_\varphi := (V_\varphi, E_\varphi)$, where V_φ is a set of all satisfying assignments of φ, that is, $V_\varphi = \{\alpha : \varphi|_\alpha = 1, \alpha \in \{0,1\}^n\}$, and $E_\varphi = \{(\alpha, \alpha') : d(\alpha, \alpha') = 1 \text{ and } \alpha, \alpha' \in V_\varphi\}$.

Definition 1. *The Boolean connectivity problem* (Conn) *is to ask whether the solution graph G_φ of a given Boolean formula φ over n variables is connected.*

Makino, Tamaki, and Yamamoto [6] showed that Conn 3-Horn is coNP-complete, thus the following lemma holds.

Lemma 1 ([6]). Conn k-Horn *is* coNP*-complete when $k \ge 3$.*

Given two assignments $\alpha, \alpha' \in \{0,1\}^n$, we denote $\alpha \le \alpha'$ as the coordinate-wise partial order if $\alpha_i \le \alpha'_i$ for each $1 \le i \le n$. A satisfying assignment α for a given formula φ is *locally minimal* if α has no neighboring satisfying assignment α' with $d(\alpha, \alpha') = 1$ and $\alpha' \le \alpha$. For $v, v' \in V_\varphi$, a *monotone path* from v to v' is a path in G_φ, $v \to u_1 \to \cdots \to u_r \to v'$ such that $v \le u_1 \le \cdots \le u_r \le v'$. Gopalan, Kolaitis, Maneva, and Papadimitriou [5] showed a notable connection between locally minimal satisfying assignments and the connectivity of Horn formulas.

Lemma 2 ([5]). *Given a Horn formula φ, every connected component of G_φ has a unique locally minimal satisfying assignment. Moreover, there exists a monotone path from the locally minimal satisfying assignment to each satisfying assignment in the same connected component.*

The following corollary follows from that any Horn formula without unit clauses over n variables has an all-zero satisfying assignment (0^n) and Lemma 2.

Corollary 1 ([6]). *Given a Horn formula φ without unit clauses, the solution graph G_φ is connected if and only if no locally minimal non-zero satisfying assignment exists.*

Let $P(C)$ (resp. $N(C)$) denote the set of variables that appear in positive (resp. negative) literals in a clause C of φ. Makino et al. [6] introduced the following Boolean formula Φ_φ.

$$\Phi_\varphi = \varphi \wedge \bigwedge_{i=1}^{n} D_i, \tag{1}$$
$$D_i = \overline{x_i} \vee \bigvee_{C \in \varphi : P(C) = \{x_i\}} \bigwedge_{y \in N(C)} y.$$

For example, given a Horn formula

$$\varphi = (x_1 \vee \overline{x_2} \vee \overline{x_3})(x_1 \vee \overline{x_3} \vee \overline{x_4})(\overline{x_2} \vee \overline{x_3} \vee \overline{x_4})(\overline{x_2} \vee x_3 \vee \overline{x_4})(\overline{x_1} \vee \overline{x_3} \vee x_4),$$

then

$$D_1 = \overline{x_1} \vee x_2 x_3 \vee x_3 x_4, \ D_2 = \overline{x_2}, \ D_3 = \overline{x_3} \vee x_2 x_4, \ D_4 = \overline{x_4} \vee x_1 x_3.$$

Note that each D_i is either a unit clause with a negative literal or a conjunction of one negative literal and disjunctions of positive literals. The satisfiability of Φ_φ is strongly related to the connectivity of the solution graph of a Horn formula φ because a satisfying assignment of Φ_φ is a locally minimal satisfying assignment of φ.

Lemma 3 ([6]). *Given a Horn formula φ without unit clauses over n variables, the solution graph G_φ is disconnected if and only if Φ_φ has a non-zero satisfying assignment.*

A *restraint clause* is a clause with only negative literals, and a *restraint set* is a set of variables included in restraint clauses. An *implication clause* is a clause that has one positive literal and one or more negative literals. A Boolean variable x is said to be *implied* by a set of variables U if setting all variables in U to true forces x to be true. A *self-implicating set* U is a set of variables such that every $x \in U$ is implied by $U \setminus \{x\}$. We also say that U is a *maximal self-implicating set* if U is a set of all variables implied by U. For maximal self-implicating sets, there exists a relation between locally minimal satisfying assignments and maximal self-implicating sets.

Lemma 4 ([13]). *For every Horn formula φ without positive unit clauses, there is a bijection correlating each connected component φ_i with a maximal self-implicating set U_i containing no restraint set, i.e., U_i consists of the variables assigned true in the minimum assignment of φ_i (the 'lowest' component is correlated with the empty set).*

The following corollary follows from Lemma 4 and Corollary 1.

Corollary 2 ([13]). *The solution graph of a Horn formula φ without positive unit clauses is disconnected if and only if φ has a non-empty maximal self-implicating set containing no restraint set.*

3 PPZ-based Algorithm for Conn k-Horn

We present an exact algorithm for Conn k-Horn based on the deterministic PPZ algorithm [9]. Before presenting our new algorithm, we describe the notations and behaviors of the deterministic PPZ algorithm, referred to as DetPPZ. Let φ be a k-CNF formula with n variables and m clauses, and let α be a satisfying assignment. A clause $C_{(\alpha,i)}$ is a *critical clause* for the variable x_i at the solution α if $C_{(\alpha,i)}$ is to be false by flipping α_i. A variable x_i is a *critical variable* if there exists a critical clause $C_{(\alpha,i)}$. The DetPPZ first checks whether there exists a satisfying assignment of φ to check all assignments with Hamming weight at most ϵn where $0 < \epsilon < 1/2$. If there is no such satisfying assignment, the DetPPZ tries to find a locally minimal satisfying assignment α with Hamming weight at least ϵn. Note that φ is falsified by any assignment obtained by flipping any 1 in α to 0. Thus, α must have at least ϵn critical variables. Let $\sigma = (\sigma_1, \ldots, \sigma_n)$ be a permutation of $\{1, \ldots, n\}$. The DetPPZ picks up a permutation σ in a 'good' small sample space and branches by assigning a 0/1 value to x_{σ_i} according to the order in σ. If any critical variable appears last among the variables in the corresponding critical clause, then it can be assigned correctly. It helps to reduce the number of branches in the algorithm. Modifying the DetPPZ such that it does not halt even if it finds a satisfying assignment with Hamming weight at most ϵn, the DetPPZ can find all locally minimal satisfying assignments. From Corollary 1, Conn k-Horn can be solved by checking whether there exists some locally minimal satisfying assignment except for 0^n assignment. Thus, by combining the (modified) DetPPZ and the procedure checking locally minimality, we solve Conn k-Horn in the same running time as the DetPPZ. We describe our algorithm, PPZ-Conn-kHorn, in Algorithm 1.

The following 'good' sample space of permutations using k-wise independent variables is useful.

Lemma 5 ([9]). *One can construct the set of permutations S of size $O(n^{3k})$ that satisfies the following condition: for any set Y of up to k variables, any variable $y \in Y$, and for a randomly chosen permutation from S, the probability that y appears last among the variables in Y is at least $1/|Y| - 1/n$.*

The following lemma immediately follows from the proof of correctness of the deterministic PPZ algorithm shown in [9]. We provide almost the same proof for self-containedness in this paper.

Lemma 6. *Let S be the set of permutations constructed by Lemma 5. Let α be an arbitrary satisfying assignment with at least ϵn critical clauses. There exists a permutation in S that produces α.*

Proof. Let α be a satisfying assignment with at least ϵn critical clauses. We now choose a permutation σ from S at random. For each critical clause at α, the probability that the critical variable x appears last in σ among the variables in the clause is at least $1/k - 1/n$ by Lemma 5. Since the number of critical clauses is at least ϵn, the expected number of times this event appears is at least $\epsilon n(1/k - 1/n) = \epsilon n/k - \epsilon$, and there exists some σ in S that achieves at least the expectation. With

Algorithm 1: PPZ-Conn-kHorn(φ)

```
1  Construct the set of permutations S in Lemma 5
2  for all assignments α with at most ϵn 1's without all 0's do
3      if α satisfies φ and α is locally minimal then
4          return NO
5  for all permutations σ in S do
6      for all strings s of n(1 − ϵ/k) + 1 bits do
7          for i = 1 to n do
8              Let x_j be the i-th variable according to σ (i.e., j = σ_i).;
9              if there exists a unit clause x_j or ¬x_j then
10                 set x_j to make that clause true
11             else if there exists an unused bit from s then
12                 set x_j equal to the next unused bit from s
13             else
14                 go to the next string in the for loop of lines 6–17.
15                                                  ▷ there are no unused bits from s
16         if φ is satisfiable and the satisfying assignment is locally minimal then
17             return NO
18 return YES
```

respect to such σ, we assign at most $n - (\epsilon n/k - \epsilon) < n(1 - \epsilon/k) + 1$ variables to satisfy φ. Thus, by checking all strings of $n(1-\epsilon/k)+1$ bits, the permutation σ can produce a satisfying assignment α. □

We next show that the locally minimality of satisfying assignments can be efficiently checked.

Lemma 7. *Let α be a satisfying assignment of φ with Hamming weight w. One can check whether α is locally minimal in $O(mw + n)$ time.*

Proof. From the definition of locally minimality, if any satisfying assignment α of φ is locally minimal, then any assignment α' obtained by flipping any 1's bit is not a satisfying assignment of φ. It can be done by checking whether α' satisfies φ. Since the Hamming weight of α is w, we can check the locally minimality of α in $O(mw + n)$ time. □

We present the main lemma, which establishes the correctness of our algorithm.

Lemma 8. *The deterministic PPZ algorithm can find all locally minimal satisfying assignments of φ.*

Proof. The deterministic PPZ algorithm first checks all assignments with Hamming weight at most ϵn. Hence, any satisfying assignment α with Hamming weight at most ϵn can be found. We can check whether α is locally minimal from Lemma 7.

We next consider that any locally minimal satisfying assignment α has Hamming weight at least ϵn. Since α is a locally minimal satisfying assignment and has at least ϵn 1's, there exist at least ϵn critical clauses at α. From Lemma 6, α must be produced by some permutation in the sample space S constructed in the algorithm. The algorithm checks all permutations in S, then it can find α. We can check whether α is truly locally minimal from Lemma 7. □

We are now ready to prove Theorem 1.

Theorem 1. *Given a k-Horn formula over n variables, there exists a deterministic $O^*(2^{(1-1/2k)n})$ time and polynomial-space algorithm for* Conn k-Horn.

Proof. We can solve Conn k-Horn by checking whether there exists some locally minimal non-zero satisfying assignment from Corollary 1. Lemma 8 shows that the deterministic PPZ algorithm can find all locally minimal satisfying assignments. Thus, PPZ-Conn-kHorn in Algorithm 1 solves

Algorithm 2: COUNT-CC-kHORN(φ)

```
1  count ← 0
2  Construct the set of permutations S in Lemma 5
3  for all assignments α with at most ϵn 1's do
4      if α satisfies φ, and α is locally minimal and has never been produced then
5          count ← count+1
6  for all permutations σ in S do
7      for all strings s of n(1 − ϵ/k) + 1 bits do
8          α ← 0^n
9          for i = 1 to n do
10             Let x_j be the i-th variable according to σ (i.e., j = σ_i).
11             if there exists a unit clause x_j or ¬x_j then
12                 set α_j to make that clause true
13             else if there exists an unused bit from s then
14                 set α_j equal to the next unused bit from s
15             else
16                 go to the next string in the for loop of lines 6–17.
17                                                  ▷ there are no unused bits from s
18         if φ is satisfiable, and the satisfying assignment α is locally minimal and has never been
             produced then
19             count ← count+1
20 return count
```

CONN k-HORN correctly. The deterministic PPZ algorithm runs in $O^*(2^{(1-1/2k)n})$ time and polynomial space. Checking locally minimality can be done $O(mn)$ time for each satisfying assignment from Lemma 7. Thus, our algorithm runs in $O^*(2^{(1-1/2k)n})$ time and polynomial space. □

By adding the procedure of counting the number of distinct locally minimal satisfying assignments (see Algorithm 2), we obtain the following Corollary.

Corollary 3. *Counting the number of connected components in the solution graph of a given k-Horn formula over n variables can be done in deterministic $O^*(2^{(1-1/2k)n})$ time and $O^*(2^{(1-1/2k)n})$ space.*

4 Complexity on the Boolean Connectivity of k-Horn Formulas with Bounded Variable Occurrence

We investigate the complexity of CONN k-HORN with bounded variable occurrences. In Section 4.1, we present a polynomial-time algorithm for CONN k-HORN-2, where each variable appears at most twice. In Section 4.2, we provide a polynomial-time algorithm for CONN Ek-HORN-3, where each clause has exactly k literals and each variable appears at most k times. In Section 4.3, we show the coNP-completeness of CONN 3-HORN-E3, where each variable appears exactly three times.

4.1 Polynomial-time Algorithm for CONN k-HORN-2

We denote φ as a k-Horn formula over n variables. If the solution graph G_φ of φ is connected, then there exists a unique locally minimal satisfying assignment 0^n from Corollary 1. Therefore, if φ has a variable x_i appearing only as a negative literal, we can show that the solution graph G_φ is connected if and only if the solution graph $G_{\varphi|_{x_i=0}}$ is connected, where $\varphi|_{x_i=0}$ denotes φ after assigning 0 to x_i. This enables us to obtain a k-Horn formula in which each variable appears at least once as a positive literal. Therefore, in the following, we can consider an instance for CONN k-HORN-2 as a k-Horn formula over n variables where each variable appears at most twice and each variable appears at least once as a positive literal. We also show that if there is a variable x_i appearing in the clause $(x_i \vee \overline{x_i})$ in φ, then it does not appear in any other clause since each variable of φ appears at most twice.

Therefore, we can eliminate the variable x_i from φ since the clause $x_i \vee \overline{x_i}$ is satisfied regardless of the value of x_i. Throughout this section, we assume that any instance of Conn k-Horn-2 contains no such variables. We now prove two auxiliary lemmas for the polynomial-time algorithm for Conn k-Horn-2.

Lemma 9. *Let φ be a k-Horn formula over n variables without unit clauses such that each variable appears at most twice in total, and at least once as a positive literal. Then, φ is a 2-Horn formula.*

Proof. Since each variable appears at most twice in φ, the total number of literals in φ is at most $2n$. Also, φ has at least n implication clauses since each variable appears at least once as a positive literal. We can show that the total number of literals in φ is at least $2n$ since φ has no unit clauses. Therefore, the total number of literals in φ is $2n$. This means that the length of each clause in φ is two since φ has no unit clause. This completes the proof. □

Lemma 10. *Let φ be a k-Horn formula over n variables without unit clauses such that each variable appears at most twice in total, and at least once as a positive literal. Then, the solution graph G_φ is disconnected.*

Proof. From the lemma 9, φ is a 2-Horn formula. Moreover, from the proof of Lemma 9, the total number of literals in φ is $2n$ and φ has at least n implication clauses. Therefore, we can show that φ has exactly n implication clauses and there exists no restraint clauses. This means that each clause has exactly one positive literal and exactly one negative literal. Hence, the assignment α' assigned the value 1 to all variables in φ is a satisfying assignment. It also holds that any assignment obtained by flipping any 1's bit of α' is not a satisfying assignment of φ. This concludes that α' is a locally minimal satisfying assignment. This completes the proof since Corollary 1 holds. □

We present the polynomial-time algorithm based on Lemma 10.

Theorem 3. Conn k-Horn-2 *can be solved in polynomial time when each variable appears at most twice.*

Proof. If a given k-Horn formula contains some unit clauses, then we repeatedly apply unit propagation until no unit clauses remain. Therefore, we assume that a given Horn formula contains no unit clauses. For a given k-Horn formula φ over n variables, if there exist variables that appear only as negative literals, we set these variables to 0. We repeat this procedure until there are no such variables in φ. Then, if all variables are assigned the value 0 in this procedure, this means that the solution graph G_φ is connected from Corollary 1 since φ has no locally minimal non-zero satisfying assignments. Otherwise, each variable in the remaining Horn formula appears at most twice in total, and at least once as a positive literal. This means that the solution graph of the remaining formula is disconnected from Lemma 10.

We now estimate the running time of this algorithm. Setting variables appearing only as negative literals to 0 can be done in $O(n)$ time since the number of clauses in φ is $O(n)$. We repeat this procedure at most n times. Hence, our algorithm runs in time $O(n^2)$. □

4.2 Polynomial-time Algorithm for Conn Ek-Horn-k

For clarity of exposition, we consider Conn E3-Horn-3, where each clause has exactly three literals and each variable appears at most three times. For a set of variables X and a CNF formula φ, we denote by $C_{\varphi[X]}$ the set of all clauses in φ that consist of only variables in X.

Lemma 11. *Let φ be an instance of* Conn E3-Horn-3 *over $X = \{x_1, x_2, \ldots, x_n\}$. The solution graph G_φ is disconnected if and only if there exists a partition of the set X into two sets U and $X \setminus U$ and a partition of the set $C_{\varphi[X]}$ of clauses of φ into two sets $C_{\varphi[U]}$ and $C_{\varphi[X\setminus U]}$ such that U is a non-empty set of variables which appear once as a positive literal.*

Proof. We first prove the if-part, and assume that a variable set U satisfies the if-condition. We show that U is a non-empty maximal self-implicating set containing no restraint set; then G_φ is disconnected from Corollary 2. The number of implication clauses in $C_{\varphi[U]}$ is $|U|$ since each variable in U appears once as a positive literal. These implication clauses have $3|U|$ literals since the length of each clause in $C_{\varphi[U]}$ is three, and then the total number of literals of clauses in $C_{\varphi[U]}$ is at least $3|U|$. However, the total number of occurrences of variables in U is at most $3|U|$ since each variable appears at most three times. These lead to the total number of literals in $C_{\varphi[U]}$ being exactly $3|U|$. Thus, all literals in $C_{\varphi[U]}$ appear in implication clauses in $C_{\varphi[U]}$, i.e., there is no restraint set in U. For any implication clause C, the positive literal x in C is assigned true when the other negative

literal(s) are assigned false. This implies that every $x \in U$ is implied by $U \setminus \{x\}$. In addition, any variable in any clause in $C_{\varphi[X \setminus U]}$ cannot be implied by U since $U \cap (X \setminus U) = \emptyset$. Hence, U is a non-empty maximal self-implicating set and contains no restraint set. This concludes the proof of the if-part.

We next prove the only-if part. Assume that G_φ is disconnected. From Corollary 2, φ has a non-empty maximal self-implicating set U containing no restraint set. This implies that every variable in U appears as a positive literal at least once. We claim that the number of implication clauses with only variables in U is $|U|$. There exist at least $|U|$ implication clauses in $C_{\varphi[X]}$ since U is a self-implicating set, i.e., there exists a clause $u \vee \overline{x} \vee \overline{y}$ for every $u \in U$. Since the total number of occurrences of variables in U is at most $3|U|$ and the length of each clause is three, there exist at most $|U|$ implication clauses with only variables in U. If the number of such implication clauses with only variables in U is at most $|U| - 1$, there exists some $u \in U$ appearing in an implication clause as positive literals with at least one variable x in $X \setminus U$. We denote $(u \vee \overline{x} \vee \overline{y})$ as such a clause. In this case, u is implied by x and y. This contradicts the fact that U is a maximal self-implicating set. Thus, the number of implication clauses with only variables in U is $|U|$, and this implies that every variable in U appears as a positive literal only once. The number of literals of these clauses is $3|U|$, and it equals the maximum total number of occurrences of variables in U. This implies that the pair $(U, X \setminus U)$ is a partition of X, and the pair of $(C_{\varphi[U]}, C_{\varphi[X \setminus U]})$ is a partition of $C_{\varphi[X]}$. This concludes the proof of the only-if part. □

Now, we present a polynomial-time algorithm based on Lemma 11.

Theorem 4. Conn 3-Horn *is solvable in polynomial time when each clause has length exactly three and each variable appears at most three times.*

Proof. A given 3-Horn formula φ over $X = \{x_1, x_2, \ldots, x_n\}$ has at most $3n$ literals and n clauses since the length of each clause is three and each variable appears at most three times. We find a non-empty variable set that satisfies Lemma 11 to determine the connectivity of the solution graph of φ.

We first enumerate the non-empty variable set U such that $C_{\varphi[X]} = C_{\varphi[U]} \cup C_{\varphi[X \setminus U]}$ and $C_{\varphi[U]} \cap C_{\varphi[X \setminus U]} = \emptyset$ as follows. We construct the graph $G = (X, E)$, where $E = \{(x_i, x_j) : C_{\varphi[X]}$ has a clause containing $x_i, x_j \in X\}$. It is easy to see that the variable set U' corresponding to the vertices in any connected component of G constructs $C_{\varphi[U']}$. Thus, we can enumerate the non-empty variable sets to enumerate the connected components of G. We use the depth-first search for G to enumerate all the connected components. It remains to check whether each U' is a set of variables that appear once as a positive literal. If at least one such U' exists, the solution graph is disconnected. Otherwise, the solution graph is connected, as shown in Lemma 11.

We now estimate the running time of this algorithm. Constructing the graph G can be done in $O(n)$ time since the number of edges is at most $3n$. Enumerating the non-empty variable set U' can be done in time $O(n)$ by the depth-first search. Note that the number of non-empty variable sets is $O(n)$. We can check whether each U' is a set of variables that appear once as a positive literal in $O(n)$ by checking the clauses. Hence, our algorithm runs in time $O(n^2)$. □

We can show Corollary 4 since the discussion in Lemma 11 also holds for $k \geq 3$.

Corollary 4. *Let φ be an instance of* Conn Ek-Horn-k *over $X = \{x_1, x_2, \ldots, x_n\}$. The solution graph G_φ is disconnected if and only if there exists a partition of the set X into two sets U and $X \setminus U$ and a partition of the set $C_{\varphi[X]}$ of clauses of φ into two sets $C_{\varphi[U]}$ and $C_{\varphi[X \setminus U]}$ such that U is a non-empty set of variables which appear once as a positive literal.*

Gopalan et al. [5] showed that Conn 2-CNF is solvable in polynomial time with no bounded variable occurrence. The proof of Theorem 4 also holds for $k \geq 3$ from Corollary 4. Therefore, we can show Corollary 5.

Corollary 5. *For $k \geq 2$,* Conn Ek-Horn-k *is solvable in polynomial time.*

We can show the following corollary from Corollary 4.

Corollary 6. *For $k \geq 3$, given a k-Horn formula φ, where each clause length and each variable occurrence are exactly k, the number of connected components in the solution graph of φ is at most $2^{\lfloor n/k \rfloor}$.*

Proof. Let φ be an instance of Conn k-Horn with each clause length exactly k and each variable occurrence exactly k, where the solution graph of φ is disconnected. A variable set of φ is denoted by X. Then, there exists at least one variable set $U \subseteq X$ such that $C_{\varphi[U]}$ and $C_{\varphi[X \setminus U]}$ partition $C_{\varphi[X]}$ and U is a non-empty set of variables that appear once as a positive literal. Now, we assume that

φ has m variable sets U_i $(1 \leq i \leq m)$ and a variable set V such that $C_{\varphi[X]} = \bigcup_{i=1}^{m} C_{\varphi[U_i]} \cup C_{\varphi[V]}$ and $\bigcup_{i=1}^{m} U_i \cup V = X, U_i \cap V = \emptyset$, and $U_i \cap U_j = \emptyset$ for any i, j $(i \neq j)$, where U_i is a non-empty set of variables which appear once as a positive literal. Then, we can construct all locally minimal satisfying assignments by setting the value 0 or 1 to all variables for each U_i $(1 \leq i \leq m)$ and setting the value 0 to all variables in V from Lemma 4. Therefore, there exist 2^m locally minimal satisfying assignments of φ. Moreover, m is a multiple of k since the length of each clause and the number of occurrences of each variable are exactly k. Hence, the maximum value of m is $\lfloor n/k \rfloor$. □

4.3 coNP-completeness of Conn 3-Horn-E3

We first prove the coNP-completeness of Conn 3-Horn-3, where each variable appears at most three times, and we then modify the instance so that each variable appears exactly three times. Instead of proving coNP-completeness of Conn 3-Horn-3, we show NP-completeness of the complement problem of Conn 3-Horn-3 (DisConn 3-Horn-3) that asks whether the solution graph of a given 3-Horn formula with each variable appearing at most three times is disconnected. The proof is based on a polynomial-time reduction from Monotone NAE E3-SAT-E4, which is NP-complete, shown in [3]. An *NAE-satisfying assignment* is a satisfying assignment in which each clause has at least one literal assigned the value 0 and at least one literal assigned the value 1. Given a Monotone 3-CNF formula ϕ with n variables and m clauses in which each clause has exactly three literals and each variable appears exactly four times, Monotone NAE E3-SAT-E4 asks whether there exists an NAE-satisfying assignment of ϕ. Our reduction utilizes the framework of Schwerdtfeger [13].

We give the construction of a 3-Horn formula from an instance of Monotone NAE E3-SAT-E4. Then, the 3-Horn formula satisfies that each clause in the instance of Monotone NAE E3-SAT-E4 has at least one literal assigned the value 0 and at least one literal assigned the value 1. Also, each variable in the 3-Horn formula appears at most three times. Let $X = \{x_1, x_2, \ldots, x_n\}$ be a set of variables. Let ϕ be an instance of Monotone NAE E3-SAT-E4 over X with m clauses. Let $C_\phi = \{C_1, C_2, \ldots, C_m\}$ be the set of clauses of the formula ϕ. For simplicity, if the variable x_j appears in the clause C_i, we write $x_j \in C_i$. We first replace the k-th occurrence of $x_j \in X$ with a new variable $x_{j,k}$, and denote by X_ϕ the set of new variables created from X. We construct a 3-Horn formula $\varphi_\phi = \varphi_1 \wedge \varphi_2 \wedge \varphi_3 \wedge \varphi_4$.

The formula φ_1 over X_ϕ is as follows:

$$\varphi_1 = \bigwedge_{i=1}^{m} \bigvee_{x_{j,k} \in C_i} \overline{x_{j,k}}.$$

It ensures that each clause in ϕ has at least one literal assigned the value 0.

The formula φ_2 is a conjunction of $\varphi_{2,i}$ corresponding to each $C_i \in C_\phi$ which contains $x_{j,a}, x_{k,b}, x_{\ell,c}$. It ensures that there exists a bijection between the set of NAE-satisfying assignments of ϕ and the set of locally minimal non-zero satisfying assignments of φ_ϕ. Let $X'_\phi = \{x'_{i,j} : 1 \leq i \leq n, 1 \leq j \leq 4\}$, $Y = \{y_{i,j} : 1 \leq i \leq m, 1 \leq j \leq 3\}$, $Y' = \{y'_{i,j} : 1 \leq i \leq m, 1 \leq j \leq 3\}$, $P = \{p_{i,j} : 1 \leq i \leq m, 1 \leq j \leq n\}$, and $Q = \{q_{i,j} : 1 \leq i \leq m, 1 \leq j \leq n\}$. The formula φ_2 over $X_\phi \cup X'_\phi \cup Y \cup Y' \cup P \cup Q$ is as follows:

$$\begin{aligned}\varphi_2 = \bigwedge_{i=1}^{m} \varphi_{2,i} = \bigwedge_{i=1}^{m} ((&\overline{y_{i,1}} \vee p_{i,j})(\overline{p_{i,j}} \vee \overline{x_{j,a}} \vee q_{i,j})(\overline{q_{i,j}} \vee x'_{j,a})(\overline{q_{i,j}} \vee y'_{(i \bmod m)+1,1}) \\ &\wedge (\overline{y_{i,2}} \vee p_{i,k})(\overline{p_{i,k}} \vee \overline{x_{k,b}} \vee q_{i,k})(\overline{q_{i,k}} \vee x'_{k,b})(\overline{q_{i,k}} \vee y'_{(i \bmod m)+1,2}) \\ &\wedge (\overline{y_{i,3}} \vee p_{i,\ell})(\overline{p_{i,\ell}} \vee \overline{x_{\ell,c}} \vee q_{i,\ell})(\overline{q_{i,\ell}} \vee x'_{\ell,c})(\overline{q_{i,\ell}} \vee y'_{(i \bmod m)+1,3})).\end{aligned}$$

The formula φ_3 is a conjunction of $\varphi_{3,j}$ corresponding to each $x_j \in X$. It ensures that $x_{j,1} = x_{j,2} = x_{j,3} = x_{j,4}$ for each j $(1 \leq j \leq n)$. Let $U = \{u_{j,k} : 1 \leq j \leq n, 1 \leq k \leq 4\}$ and $D_X = \{d_j : 1 \leq j \leq n\}$. The formula φ_3 over $D_X \cup X_\phi \cup X'_\phi \cup U$ is as follows:

$$\begin{aligned}\varphi_3 = \bigwedge_{j=1}^{n} \varphi_{3,j} = \bigwedge_{j=1}^{n} ((&u_{j,1} \vee \overline{x'_{j,1}} \vee \overline{x'_{j,2}})(u_{j,2} \vee \overline{x'_{j,3}} \vee \overline{x'_{j,4}})(d_j \vee \overline{u_{j,1}} \vee \overline{u_{j,2}}) \\ &\wedge (u_{j,3} \vee \overline{d_j})(u_{j,4} \vee \overline{d_j})(x_{j,1} \vee \overline{u_{j,3}})(x_{j,2} \vee \overline{u_{j,3}})(x_{j,3} \vee \overline{u_{j,4}})(x_{j,4} \vee \overline{u_{j,4}})).\end{aligned}$$

Recall that a self-implicating set S is a set of variables such that every $x \in S$ is implied by $S \setminus \{x\}$. The following four claims establish properties of a non-empty maximal self-implicating set for the

formula above. All proofs proceed by repeatedly invoking the argument that the existence of a clause in the formula guarantees that at least one variable appearing in the clause belongs to a non-empty maximal self-implicating set.

Claim 1. *Let S be an arbitrary non-empty maximal self-implicating set. Then, the following properties hold:*

(a) If S contains at least one $x_{j,k}$ $(1 \leq k \leq 4)$, then S contains all variables appearing in $\varphi_{3,j}$,

(b) If S contains d_j, then S contains all $x_{j,k}$ and at least one $x'_{j,k}$,

(c) If S contains at least one $u_{j,k}$, then S contains at least one $x'_{j,k}$.

Proof. We show each property.

(a) Without loss of generality, we assume that S contains $x_{j,1}$. The positive literal $x_{j,1}$ appears only in the clause $(x_{j,1} \vee \overline{u_{j,3}})$ of $\varphi_{3,j}$, then S contains $u_{j,3}$ to imply $x_{j,1}$ by variables in $S \setminus \{x_{j,1}\}$. By a similar argument and the clauses $(d_j \vee \overline{u_{j,3}}), (x_{j,2} \vee \overline{u_{j,3}}), (d_j \vee \overline{u_{j,1}} \vee \overline{u_{j,2}}), (u_{j,1} \vee \overline{x'_{j,1}} \vee \overline{x'_{j,2}})$, and $(u_{j,2} \vee \overline{x'_{j,3}} \vee \overline{x'_{j,4}})$, S contains the variables $d_j, x_{j,2}, u_{j,1}, u_{j,2}, x'_{j,1}, x'_{j,2}, x'_{j,3}$, and $x'_{j,4}$. From the maximality of S and the clause $(u_{j,4} \vee \overline{d_j})$, S contains $u_{j,4}$. It means that S contain $x_{j,3}$ and $x_{j,4}$ since $u_{j,4}$ implies both two variables from the clauses $(x_{j,3} \vee \overline{u_{j,4}}), (x_{j,4} \vee \overline{u_{j,4}})$. Therefore, S contain all variables appearing in $\varphi_{3,j}$.

(b) We first show that S contains all $x_{j,k}$. If S contains d_j, then the variables $u_{j,3}$ and $u_{j,4}$ are contained from the maximality of S and the clauses $(u_{j,3} \vee \overline{d_j}), (u_{j,4} \vee \overline{d_j})$. By a similar argument and the clauses $(x_{j,1} \vee \overline{u_{j,3}}), (x_{j,2} \vee \overline{u_{j,3}}), (x_{j,3} \vee \overline{u_{j,4}}), (x_{j,4} \vee \overline{u_{j,4}})$, S contains $x_{j,1}, x_{j,2}, x_{j,3}$ and $x_{j,4}$. We next show that S contains at least one $x'_{j,k}$. By a similar argument of (a) and the clauses $(d_j \vee \overline{u_{j,1}} \vee \overline{u_{j,2}}), (u_{j,1} \vee \overline{x'_{j,1}} \vee \overline{x'_{j,2}})$, and $(u_{j,2} \vee \overline{x'_{j,3}} \vee \overline{x'_{j,4}})$, S contains the variables $u_{j,1}, u_{j,2}, x'_{j,1}, x'_{j,2}, x'_{j,3}$, and $x'_{j,4}$ if S contains d_j. Hence, S contains at least one $x'_{j,k}$.

(c) We consider two cases: (i) S contains $u_{j,1}$ or $u_{j,2}$ (ii) S contains $u_{j,3}$ or $u_{j,4}$. In Case (i), S contains the variables $x'_{j,1}, x'_{j,2}$ or $x'_{j,3}, x'_{j,4}$ from the clauses $(u_{j,1} \vee \overline{x'_{j,1}} \vee \overline{x'_{j,2}}), (u_{j,2} \vee \overline{x'_{j,3}} \vee \overline{x'_{j,4}})$. In Case (ii), S contains d_j from the clauses $(u_{j,3} \vee \overline{d_j}), (u_{j,4} \vee \overline{d_j})$. Therefore, S contains at least one $x'_{j,k}$ from (b).

□

The formula φ_4 is a conjunction of $\varphi_{4,i}$ corresponding to each $C_i \in C_\phi$. It ensures that each clause in ϕ has at least one literal assigned the value 1. Let $V = \{v_{i,j} : 1 \leq i \leq m, 1 \leq j \leq 4\}$ and $E_Y = \{e_i : 1 \leq i \leq m\}$. The formula φ_4 over $E_Y \cup Y \cup Y' \cup V$ is as follows:

$$\varphi_4 = \bigwedge_{i=1}^{m} \varphi_{4,i} = \bigwedge_{i=1}^{m} ((v_{i,1} \vee \overline{y'_{i,1}})(v_{i,1} \vee \overline{y'_{i,2}})(v_{i,2} \vee \overline{y'_{i,3}})(v_{i,2} \vee \overline{v_{i,1}})(e_i \vee \overline{v_{i,2}})(v_{i,3} \vee \overline{e_i}) \\ \wedge (v_{i,4} \vee \overline{v_{i,3}})(y_{i,1} \vee \overline{v_{i,4}})(y_{i,2} \vee \overline{v_{i,4}})(y_{i,3} \vee \overline{v_{i,3}})).$$

From the construction of $\varphi_{4,i}$, we can get the following observation.

Claim 2. *Let S be an arbitrary non-empty maximal self-implicating set. Then, the following properties hold:*

(a) If S contains at least one $y_{i,k}$, then S contains e_i and at least one $y'_{i,k}$ $(1 \leq k \leq 3)$,

(b) If S contains at least one $v_{i,\ell}$ $(1 \leq \ell \leq 4)$, then S contains e_i,

(c) If S contains at least one $y'_{i,k}$, then S contains e_i,

(d) If S contains e_i, then S contains at least one $y'_{i,k}$ $(1 \leq k \leq 3)$.

Proof. We show each property.

(a) Without loss of generality, we assume that S contains $y_{i,1}$. The positive literal $y_{i,1}$ appears only in the clause $(y_{i,1} \vee \overline{v_{i,4}})$ of $\varphi_{4,i}$, then S contains $v_{i,4}$ to imply $y_{i,1}$ by variables in $S \setminus \{y_{i,1}\}$. By a similar argument and the clauses $(v_{i,4} \vee \overline{v_{i,3}}), (v_{i,3} \vee \overline{e_i})$, and $(e_i \vee \overline{v_{i,2}})$, S contains the variables $v_{i,4}, v_{i,3}, e_i$, and $v_{i,2}$. The variable $v_{i,2}$ appears in the clauses $(v_{i,2} \vee \overline{y'_{i,3}})$ and $(v_{i,2} \vee \overline{v_{i,1}})$, then S contains $y'_{i,3}$ or $v_{i,1}$. If S contains $y'_{i,3}$, then the proof is done. Otherwise, S contains $v_{i,1}$. In this case, S contains $y'_{i,1}$ or $y'_{i,2}$ to imply $v_{i,1}$ by variables in $S \setminus \{v_{i,1}\}$. Therefore, S contains at least one $y'_{i,k}$ $(1 \leq k \leq 3)$.

(b) We consider two cases: (i) S contains $v_{i,1}$ or $v_{i,2}$ (ii) S contains $v_{i,3}$ or $v_{i,4}$.

Case (i) If S contains $v_{i,2}$, then S contains e_i from the maximality of S and the clause $(e_i \vee \overline{v_{i,2}})$. By a similar argument and the clauses $(v_{i,2} \vee \overline{v_{i,1}}), (e_i \vee \overline{v_{i,2}})$, if S contains $v_{i,1}$, then $v_{i,2}$ and e_i are contained in S.

Case (ii) The set S contains e_j from the clause $(v_{i,3} \vee \overline{e_i})$ if S contains $v_{i,3}$. If S contains $v_{i,4}$, then S contains $v_{i,3}$ from the clause $(v_{i,4} \vee \overline{v_{i,3}})$. This means that S contains e_j by a similar argument.

(c) Without loss of generality, we assume that S contains $y'_{i,1}$. Then, from the maximality of S and the clauses $(v_{i,1} \vee \overline{y'_{i,1}}), (v_{i,2} \vee \overline{v_{i,1}})$, S contains $v_{i,1}$ and $v_{i,2}$. By a similar argument and the clause $(e_i \vee \overline{v_i}, \overline{2})$, S contains e_i.

(d) By a similar argument to (a) and the clause $(e_i \vee \overline{v_{i,2}})$, S contains $v_{i,2}$. The variable $v_{i,2}$ appears in the clauses $(v_{i,2} \vee \overline{y'_{i,3}})$ and $(v_{i,2} \vee \overline{v_{i,1}})$, then S contains $y'_{i,3}$ or $v_{i,1}$. If S contains $y'_{i,3}$, then the proof is done. Otherwise, S contains $v_{i,1}$. In this case, S contains $y'_{i,1}$ or $y'_{i,2}$ to imply $v_{i,1}$ by variables in $S \setminus \{v_{i,1}\}$. Therefore, S contains at least one $y'_{i,k}$ $(1 \leq k \leq 3)$.

□

Clearly, φ_ϕ is a 3-Horn formula. The number of occurrences of each $d_j \in D_X$, $x_{j,k} \in X_\phi$, $q_{i,j} \in Q$, and $v_{i,j} \in V$ is exactly three, and other variables appear at most three times. Hence, φ_ϕ is an instance of DisConn 3-Horn-3. We show the following two claims about maximal self-implicating sets of φ_ϕ.

Claim 3. *Let S be an arbitrary non-empty maximal self-implicating set of φ_ϕ. Then, S contains all $e_i \in E_Y$ and all $p_{i,j} \in P$.*

Proof. To prove this claim, we show that S contains all e_i. If S contains all e_i, then S contains all $v_{i,3}$ from the maximality of S and the clause $(v_{i,3} \vee \overline{e_i})$ of $\varphi_{4,i}$. The variables $v_{i,4}$ and $y_{i,3}$ are contained from the maximality of S and the clauses $(v_{i,4} \vee \overline{v_{i,3}}), (y_{i,3} \vee \overline{v_{i,3}})$. Then, S contains $y_{i,1}$ and $y_{i,2}$ which are implied by $v_{i,4}$ in the clauses $(y_{i,1} \vee \overline{v_{i,4}}), (y_{i,2} \vee \overline{v_{i,4}})$. Now, S contains the variables $y_{i,1}, y_{i,2}$, and $y_{i,3}$. From the clauses $(\overline{y_{i,1}} \vee p_{i,j}), (\overline{y_{i,2}} \vee p_{i,k}), (\overline{y_{i,3}} \vee p_{i,\ell})$ and the maximality of S, the variables $p_{i,j} \in P$ are contained in S. We prove that S contains all e_i in two steps: (i) S has at least one e_i (ii) If S has at least one e_i, then S contains all e_i.

We first show (i). For the sake of contradiction, we assume that there exists a non-empty maximal self-implicating set S that contains no element $e_i \in E_Y$. Then, we consider cases depending on which variable sets of φ_ϕ the non-empty maximal self-implicating set S contains at least one variable from. The formula φ_ϕ has the set of variables $X_\phi, X'_\phi, Y, Y', P, Q, U, V, D_X$, and E_Y. The formula $\varphi_{4,i}$ has the set of variables E_Y, Y, Y', and V. The formula $\varphi_{3,j}$ has the set of variables D_X, X_ϕ, X'_ϕ, and U. The remaining sets of variables in φ_ϕ are P and Q. Thus, we consider the following four cases:

1. S has at least one variable except for e_i, appearing in $\varphi_{4,i}$,
2. S has at least one variable appearing in $\varphi_{3,j}$,
3. S has at least one variable in P,
4. S has at least one variable in Q.

Case 1 The formula $\varphi_{4,i}$ has the set of variables E_Y, Y, Y', and V. If S contains at least one variable in Y, then S contains some $e_i \in E_Y$ from the property (a) in Claim 2. If S contains at least one variable in Y', then S contains some $e_i \in E_Y$ from the property (a) in Claim 2. Also, if S contains at least one variable in V, then S contains some $e_i \in E_Y$ from the property (b) in Claim 2. This contradicts the assumption.

Case 2 The formula $\varphi_{3,j}$ has the set of variables D_X, X_ϕ, X'_ϕ, and U. If S contains any variable appearing in $\varphi_{3,j}$, then S contains at least one $x'_{j,k}$ $(1 \leq k \leq 4)$ from the properties (a), (b), and (c) in Claim 1. The positive literal $x'_{j,k}$ appears only in $(\overline{q_{i,j}} \vee x'_{j,k})$ of $\varphi_{2,i}$, then S contains $q_{i,j}$ to imply $x'_{j,k}$ by variables in $S \setminus \{x'_{j,k}\}$. Therefore, since positive literal $q_{i,j}$ appears only in $(\overline{p_{i,j}} \vee \overline{x_{j,k}} \vee q_{i,j})$ of $\varphi_{2,j}$, S contains $p_{i,j}$ and $x_{j,k}$. The clause $(\overline{y_{i,a}} \vee p_{i,j})$ of $\varphi_{2,i}$ ensures that S contains $y_{i,a}$ to imply $p_{i,j}$ by variables in $S \setminus \{p_{i,j}\}$. Then, S contains e_i from the property (a) in Claim 2. This contradicts the assumption.

Case 3 and Case 4 We can consider the latter two cases as subcases of Case 2. Therefore, we can show both cases by using the same discussion as Case 2.

We next show (ii). We assume that there exists a non-empty maximal self-implicating set S which does not contain some $e_i \in E_Y$. From the above discussion, S contains at least one $e_i \in E_Y$. Then, S contains at least one $y'_{i,j}$ from the property (d) in Claim 2. Let $\ell = ((i+m-2) \bmod m)+1$ and note that $\ell = m$ if $i = 1$ and $\ell = i-1$ otherwise. From the clauses $(\overline{q_{\ell,j}} \vee y'_{i,1}), (\overline{q_{\ell,k}} \vee y'_{i,2}), (\overline{q_{\ell,a}} \vee y'_{i,3})$ of $\varphi_{4,\ell}$, S contains $q_{\ell,j}$ to imply $y'_{i,j}$ by variables in $S \setminus \{y'_{i,j}\}$. Therefore, S contains e_ℓ by a similar argument of Case 4 in (i). By repeating these arguments, all variables in E_Y are contained in S. This contradicts the assumption. Hence, all $e_i \in E_Y$ are contained in S, and this completes the proof. □

Claim 4. *Let S be an arbitrary non-empty maximal self-implicating set of φ_ϕ. Then, for each j $(1 \le j \le n)$, S contains at least one $q_{i,j} \in Q$ $(1 \le i \le m)$ if and only if S contains all variables appearing in $\varphi_{3,j}$.*

Proof. We show the following two propositions, respectively.

(i) For each j, if S contains at least one $q_{i,j}$, then S contains all variables appearing in $\varphi_{3,j}$.

(ii) For each j, if S does not contain any $q_{i,j}$ for all i, then S does not contain any variables appearing in $\varphi_{3,j}$.

Case (i) From Claim 3, S contains all e_i and all $p_{i,j}$. Since $\varphi_{2,i}$ has the clause $(\overline{p_{i,j}} \vee \overline{x_{j,a}} \vee q_{i,j})$ and S contains at least one $q_{i,j}$ and all $p_{i,j}$, we can show that S contains $x_{j,a}$ to imply $q_{i,j} \in S$ by some variables in $S \setminus \{q_{i,j}\}$. Therefore, S contains all variables which appear in $\varphi_{3,j}$ since the property (a) in Claim 1 holds.

Case (ii) From Claim 3, S contains all e_i and all $p_{i,j}$. For any j satisfying that $q_{i,j} \notin S$ for all i, S does not contain some $x_{j,a}$ from the clause $(\overline{p_{i,j}} \vee \overline{x_{j,a}} \vee q_{i,j})$ in $\varphi_{2,j}$. We now assume that S contains at least one variable, other than $x_{j,a} \notin S$, that appears in $\varphi_{3,j}$. The formula $\varphi_{3,j}$ has the sets of variables D_X, X_ϕ, X'_ϕ, and U. Thus, we consider the following four cases depending on which variable sets in $\varphi_{3,j}$ the non-empty maximal self-implicating set S contains:

(a) S has at least one variable in $X_\phi \setminus \{x_{j,a}\}$,

(b) S has at least one variable in D_X,

(c) S has at least one in U,

(d) S has at least one in X'_ϕ.

(a) If S contains some $x_{j,\ell} \in X_\phi$ other than $x_{j,a}$, then S contains all variables appearing in $\varphi_{3,j}$ from the property (a) in Claim 1. This contradicts the fact that S does not contain $x_{j,a}$.

(b) If S contains $d_j \in D_X$, then S contains some $x_{j,k}$ from the property (b) in Claim 1. This means that S contains all variables appearing in $\varphi_{3,j}$ from the property (a) in Claim 1. This contradicts the fact that S does not contain $x_{j,a}$.

(c) In this case, we consider two cases depending on which variables in U are contained in S: (1) S contains $u_{j,3}$ or $u_{j,4}$ (2) S contains $u_{j,1}$ or $u_{j,2}$.

Case (1) If S contains $u_{j,3}$ or $u_{j,4}$, then S contains d_j to imply $u_{j,3}, u_{j,4}$ from the clauses $(u_{j,3} \vee \overline{d_j}), (u_{j,4} \vee \overline{d_j})$. Therefore, we can consider this case as a subcase of Case (b), and this leads to a contradiction by a similar argument to Case (b).

Case (2) In this case, S contains at least one $x'_{j,k}$ to imply $u_{j,1}, u_{j,2}$ from the property (c) in Claim 1. Then, S contains $q_{\ell,j}$ to imply $x'_{j,k} \in S$ since positive literal $x'_{j,k}$ appears only in $(\overline{q_{\ell,j}} \vee x'_{j,k})$ of $\varphi_{2,\ell}$. By similar argument and the clause $(\overline{p_{\ell,j}} \vee \overline{x_{j,k'}} \vee q_{\ell,j})$ of $\varphi_{2,\ell}$, S contains $x_{j,k'}$. This means that S contains all variables in $\varphi_{3,j}$ from the property (a) in Claim 1. This contradicts the fact that S does not contain $x_{j,a}$.

(d) We can consider this case as a subcase of Case (c). Therefore, we can lead the contradiction by a similar argument of Case (c).

□

We now show two auxiliary Lemmas to prove the NP-hardness of DisConn 3-Horn-3.

Lemma 12. *φ_ϕ has a locally minimal non-zero satisfying assignment if ϕ has an NAE-satisfying assignment.*

Proof. Let α be an NAE-satisfying assignment of ϕ and let us construct a satisfying assignment α' of φ_ϕ from α. The formula $\varphi_{3,j}$ has the sets of variables D_X, X_ϕ, X'_ϕ and U. The formula $\varphi_{4,i}$ has the sets of variables E_Y, Y, Y' and V. The remaining sets of variables in φ_ϕ are P and Q. Then, for each j, we set the value of all variables appearing in $\varphi_{3,j}$ to the same value of $\alpha(x_j)$ in α'. For each j, we also set the value of each variable $q_{i,j} \in Q$ and each variable $y'_{i,k} \in Y'$ to the same value of $\alpha(x_j)$, where $y'_{i,k}$ shares the clause $(\overline{q_{i,j}} \vee y'_{i,k})$ with $q_{i,j}$. Moreover, we set the value of all variables in E_Y, Y, and P to 1 in α'. For each variable in V, we consider the following assignment. We set all $v_{i,2}, v_{i,3}$, and $v_{i,4}$ to the value 1 in α'. For each $v_{i,1}$, we set it to the value 1 if $y'_{i,1}$ or $y'_{i,2}$ is assigned the value 1, and 0 otherwise. Then, α' is a satisfying assignment of φ_ϕ. For each variable, there exists an implication clause such that if the variable is assigned the value 1 and appears as a positive literal, then all remaining variables in the clause are also assigned the value 1. By the definition of the implication clause, any assignment obtained by flipping any 1's bit in α' is not a satisfying assignment of φ_ϕ. This means that α' is a locally minimal non-zero satisfying assignment of φ_ϕ. □

Lemma 13. *ϕ has an NAE-satisfying assignment if φ_ϕ has a locally minimal non-zero satisfying assignment.*

Proof. Let α be a locally minimal non-zero satisfying assignment of φ_ϕ and let S be a maximal self-implicating set corresponding to α. Note that S is the set of variables assigned the value 1 in α. We now construct a satisfying assignment α' of ϕ from α by setting $\alpha(x_j)$ to 1 if $d_j \in S$ and $\alpha(x_j)$ to 0 if $d_j \notin S$ for each j $(1 \leq j \leq n)$, and show that α' is an NAE-satisfying assignment. From Claim 3, S contains all $e_i \in E_Y$ $(1 \leq i \leq m)$ and all $p_{i,j} \in P$. This means that S contains at least one $y'_{i,j'}$ $(1 \leq j' \leq 3)$ for each i from the property (d) in Claim 2. Moreover, for each i, S contains at least one $q_{i,j}$ to imply all $y'_{i,j'} \in S$ by variables in $S \setminus \{y'_{i,j'}\}$ since S is a maximal self-implicating set and each $\varphi_{2,i}$ has the clauses $(\overline{q_{i,j}} \vee y'_{(i \bmod m)+1,1})$, $(\overline{q_{i,k}} \vee y'_{(i \bmod m)+1,2})$, and $(\overline{q_{i,\ell}} \vee y'_{(i \bmod m)+1,3})$. From Claim 4, for each j such that $q_{i,j} \in S$, S contains all variables appearing in $\varphi_{3,j}$. This means that i-th clause C_i of ϕ, which has $x_{j,a}, x_{k,b}$ and $x_{\ell,c}$, has at least one literal assigned the value 1. If S contains all $q_{i,j}$ for some i, then S contains $x_{j,a}, x_{k,b}$ and $x_{\ell,c}$ to imply all $q_{i,j}$ from the clauses $(\overline{p_{i,j}} \vee \overline{x_{j,a}} \vee q_{i,j})$, $(\overline{p_{i,k}} \vee \overline{x_{k,b}} \vee q_{i,k})$, and $(\overline{p_{i,\ell}} \vee \overline{x_{\ell,c}} \vee q_{i,\ell})$ of $\varphi_{2,i}$. It means that the value of $x_{j,a}, x_{k,b}$ and $x_{\ell,c}$ is 1, and the clause $(\overline{x_{j,a}} \vee \overline{x_{k,b}} \vee \overline{x_{\ell,c}})$ of φ_1 is falsified. Thus, at least one $x_{j,a}$ must be excluded from S, and then for each i, at least one $q_{i,j}$ also must be excluded from S. This means that for each i, i-th clause C_i of ϕ has at least one literal assigned the value 0. Therefore, for each j such that $q_{i,j} \notin S$, S does not contain all variables appearing in $\varphi_{3,j}$ from Claim 4. For each j, we now set $\alpha'(x_j) = 1$ if $d_j \in S$, and set $\alpha'(x_j) = 0$ if $d_j \notin S$. Then, α' is an NAE-satisfying assignment of ϕ since each clause in ϕ has at least one literal assigned the value 0 and at least one literal assigned the value 1 in α'. This completes the proof. □

We are ready to prove the following theorem.

Theorem 5. DisConn 3-Horn-3 *is* NP*-complete.*

Proof. Let ϕ be an instance of Monotone NAE E3-SAT-E4. Let φ_ϕ be a 3-Horn formula constructed from ϕ, and Φ_{φ_ϕ} be the formula constructed from φ_ϕ. By Lemma 3, if Φ_{φ_ϕ} has a non-zero satisfying assignment, then the solution graph G_{φ_ϕ} is disconnected; i.e., a non-zero satisfying assignment of Φ_{φ_ϕ} is a certificate of disconnection for G_{φ_ϕ}. Thus, DisConn 3-Horn-4 belongs to NP. Lemma 12, Lemma 13, and Lemma 3 imply that DisConn 3-Horn-3 is NP-hard. Therefore, DisConn 3-Horn-3 is NP-complete. □

The coNP-completeness of Conn 3-Horn-3 immediately follows from Theorem 5. We then show the following Theorem.

Theorem 6. Conn 3-Horn-E3 *is* coNP*-complete.*

Proof. We prove the coNP-hardness of Conn 3-Horn-E3 by a polynomial-time reduction from Conn 3-Horn-3. We first show the construction of a 3-Horn formula with each variable appearing exactly three times from an instance of Conn 3-Horn-3. Let ϕ be an instance of Conn 3-Horn-3. Then, for every variable v which appears twice, we introduce new variables v_1, v_2 and add three clauses $(\overline{v_1} \vee \overline{v_2})(\overline{v_1} \vee \overline{v_2})(\overline{v_1} \vee \overline{v_2} \vee \overline{v})$. For each variable that appears once, we repeat the same operation until each variable appears three times. Thus, we can obtain the formula φ, which is an instance of Conn 3-Horn-E3. Then, we transform ϕ and φ to Φ_ϕ and Φ_φ for simplicity of proving the correctness of our reduction. From the construction, Φ_φ has unit clauses with negative literals of

all added variables. Therefore, Φ_φ is equal to Φ_ϕ by assigning 0 to all added variables of φ. It means that Φ_ϕ has a non-zero satisfying assignment if and only if Φ_φ has a non-zero satisfying assignment, and by Lemma 3 this completes the proof. □

By duality, coNP-completeness of the Boolean connectivity for dual-Horn formulas with the same restrictions follows from the same discussion.

5 Conclusion

We proved that Conn 3-Horn-E3 is coNP-complete. Furthermore, we presented a polynomial-time algorithm for Conn 3-Horn-2. As a result, we established the complexity boundary between P and coNP-completeness for Conn 3-Horn with bounded variable occurrences. However, this complexity boundary could not be determined in the case where each clause has exactly three literals. Although we derived a polynomial-time algorithm for Conn E3-Horn-3, it remains open whether Conn E3-Horn-E4 — namely, Conn 3-Horn in which each variable appears exactly four times and each clause has exactly three literals — is coNP-complete. We conjecture that Conn E3-Horn-E4 is coNP-complete, since Monotone NAE 3-SAT remains NP-complete even when each clause has exactly three literals and each variable appears exactly four times [3], and because the currently known complexity results for Conn 3-Horn with bounded variable occurrences mirror those for Monotone NAE 3-SAT under the same restrictions.

References

[1] Paul S. Bonsma, Amer E. Mouawad, Naomi Nishimura, and Venkatesh Raman. The Complexity of Bounded Length Graph Recoloring and CSP Reconfiguration. In *Proceedings of the 9th International Symposium on Parameterized and Exact Computation (IPEC 2014)*, volume 8894 of *Lecture Notes in Computer Science*, pages 110–121, Berlin, 2014. Springer. doi:10.1007/978-3-319-13524-3_10.

[2] Jean Cardinal, Erik D. Demaine, David Eppstein, Robert A. Hearn, and Andrew Winslow. Reconfiguration of Satisfying Assignments and Subset Sums: Easy to Find, Hard to Connect. In *Proceedings of 24th International Computing and Combinatorics Conference (COCOON 2018)*, volume 10976 of *Lecture Notes in Computer Science*, pages 365–377, Berlin, 2018. Springer. doi:10.1007/978-3-319-94776-1_31.

[3] Andreas Darmann and Janosch Döcker. On a simple hard variant of Not-All-Equal 3-Sat. *Theoretical Compututer Science*, 815:147–152, 2020. doi:10.1016/J.TCS.2020.02.010.

[4] Oya Ekin, Peter L. Hammer, and Alexander Kogan. On Connected Boolean Functions. *Discrete Applied Mathematics*, 96-97:337–362, 1999. doi:10.1016/S0166-218X(99)00098-0.

[5] Parikshit Gopalan, Phokion G. Kolaitis, Elitza N. Maneva, and Christos H. Papadimitriou. The Connectivity of Boolean Satisfiability: Computational and Structural Dichotomies. *SIAM Journal on Computing*, 38(6):2330–2355, 2009. doi:10.1137/07070440X.

[6] Kazuhisa Makino, Suguru Tamaki, and Masaki Yamamoto. On the Boolean connectivity problem for Horn relations. *Discrete Applied Mathematics*, 158(18):2024–2030, 2010. doi:10.1016/J.DAM.2010.08.019.

[7] Kazuhisa Makino, Suguru Tamaki, and Masaki Yamamoto. An exact algorithm for the Boolean connectivity problem for k-CNF. *Theoretical Computer Science*, 412(35):4613–4618, 2011. doi:10.1016/j.tcs.2011.04.041.

[8] Amer E. Mouawad, Naomi Nishimura, Vinayak Pathak, and Venkatesh Raman. Shortest Reconfiguration Paths in the Solution Space of Boolean Formulas. In *Proceedings of the 42nd EATCS International Colloquium on Automata, Languages, and Programming (ICALP 2015)*, volume 9134 of *Lecture Notes in Computer Science*, pages 985–996, Berlin, 2015. Springer. doi:10.1007/978-3-662-47672-7_80.

[9] Ramamohan Paturi, Pavel Pudlák, and Francis Zane. Satisfiability Coding Lemma. *Chicago Journal of Theoretical Computer Science*, 1999.

[10] Thomas J. Schaefer. The Complexity of Satisfiability Problems. In *Proceedings of the 10th Annual ACM Symposium on Theory of Computing (STOC 1978)*, pages 216–226, New York, 1978. ACM. doi:10.1145/800133.804350.

[11] Patrick Scharpfenecker. On the Structure of Solution-Graphs for Boolean Formulas. In *Proceedings of the 20th International Symposium (FCT 2015)*, volume 9210 of *Lecture Notes in Computer Science*, pages 118–130, Berlin, 2015. Springer. doi:10.1007/978-3-319-22177-9_10.

[12] Patrick Scharpfenecker and Jacobo Torán. Solution-Graphs of Boolean Formulas and Isomorphism. *Journal on Satisfiability, Boolean Modelling and Computation*, 10(1):37–58, 2016. doi:10.3233/SAT190113.

[13] Konrad W. Schwerdtfeger. A Computational Trichotomy for Connectivity of Boolean Satisfiability. *Journal on Satisfiability, Boolean Modelling and Computation*, 8(3/4):173–195, 2014. doi:10.3233/sat190097.